

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S.Mahammed Nayeem	Reg. No.:	192425294
Section / Batch:	AI&ML/2024	Date:	19-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I S.Mahammed Nayeem/192425294 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

S.Mahammed Nayeem

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				

Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim

To implement a Dynamic Programming algorithm to determine the optimal inventory replenishment strategy while minimizing ordering, holding, and shortage costs.

Procedure

1. Define the planning period T , maximum storage capacity K , and demand for each period.
2. Define the holding cost, shortage cost, and ordering cost.
3. Create a Dynamic Programming table to store the minimum cost for each period and inventory level.
4. Consider all possible order quantities for every inventory state.
5. Calculate the inventory after satisfying the demand.
6. Add ordering cost and either holding cost or shortage cost.
7. Store the minimum cost for each state using the recurrence relation.
8. Continue the process until the final period.
9. Find the minimum value in the final DP state.
10. Display the minimum total inventory replenishment cost.

Python Code

```
def inventory_dp(T, K, demand, hold, shortage, order_cost):
```

```
    INF = 10**18
```

```
    dp = [[INF] * (K + 1) for _ in range(T + 1)]
```

```
    dp[0][0] = 0
```

```
    for t in range(T):
```

```
        for s in range(K + 1):
```

```
            if dp[t][s] >= INF:
```

```
                continue
```

```
            for x in range(K - s + 1):
```

```
                ns = s + x - demand[t]
```

```
                cost = order_cost * x
```

```
            if ns >= 0:
```

```
                cost += hold * ns
```

```
                ns = min(ns, K)
```

```
            else:
```

```
                cost += shortage * (-ns)
```

```
                ns = 0
```

```
            dp[t + 1][ns] = min(
```

```
                dp[t + 1][ns],
```

```
                dp[t][s] + cost
```

)

```
best = min(dp[T])  
return best
```

```
T = 5  
K = 10  
demand = [2, 3, 2, 4, 1]  
hold = 1  
shortage = 5  
order_cost = 2  
  
result = inventory_dp(  
    T, K, demand, hold, shortage, order_cost  
)  
  
print(result)
```

Input

```
T = 5  
K = 10  
Demand = [2, 3, 2, 4, 1]  
Holding Cost = 1  
Shortage Cost = 5  
Ordering Cost = 2
```

Output

24

ASSESSMENT CO4-AT2-Industry Problem Solving Task

← Back

Languages: Python C C++ Java

QUESTIONS

Q1

Smart Inventory Replenishment
Optimization (Dynamic
Programming)
100 marks

1/1 answered

Q1 Smart Inventory Replenishment Optimization (Dynamic Programming)

100 marks

A retail chain must determine optimal inventory replenishment schedules to minimize holding and shortage costs. Analyze how this problem can be formulated using Dynamic Programming. Identify constraints such as demand variability, storage limits, and lead times. Design a solution using recurrence relations for optimal decision-making. Discuss implementation feasibility in supply chain systems. Evaluate algorithm efficiency and scalability. Interpret performance in terms of cost savings and inventory optimization.

Your Code

```
1 def inventory_dp(T, K, demand, hold, shortage, order_cost):
2     INF = 10**18
3     dp = [[INF]*(K+1) for _ in range(T+1)]
4     dp[0][0] = 0
5     for t in range(T):
6         for s in range(K+1):
7             if dp[t][s] == INF:
8                 continue
9             for x in range(K - s + 1):
10                ns = s + x - demand[t]
11                cost = order_cost*x
12                if ns >= 0:
13                    cost += hold*ns
14                    ns = min(ns, K)
15                else:
16                    cost += shortage*(-ns)
17                    ns = 0
18                dp[t+1][ns] = min(dp[t+1][ns], dp[t][s] + cost)
19     best = min(dp[T])
20     return best
21
22 T = 5
23 K = 10
24 demand = [2, 3, 2, 4, 1]
25 hold = 1
26 shortage = 5
27 order_cost = 2
```

Input

stdin input

Output

24

Result

The Dynamic Programming algorithm successfully determines the minimum total inventory replenishment cost as 24 for the given demand, storage, holding, shortage, and ordering constraints.